Tu travailles sur Hardgraft.

Les documents GAME DESIGN SOURCE OF TRUTH et PLAN CODEX sont les références principales du projet. Lis-les intégralement avant toute modification.

Règle principale

Ne cherche jamais à développer Hardgraft dans son ensemble en une seule fois.

Le projet doit progresser par petites couches fonctionnelles, testables et jouables.

La priorité est toujours :

faire fonctionner proprement la couche actuelle avant d'ajouter la suivante.

Une fonctionnalité future décrite dans le Game Design Document n'est pas une demande d'implémentation immédiate.

Ordre de travail obligatoire

Avant de commencer une tâche :

1. Lire les documents de référence.
2. Inspecter le code réellement présent dans la branche.
3. Identifier les systèmes déjà fonctionnels.
4. Identifier les fichiers concernés.
5. Identifier les tests existants.
6. Déterminer la modification minimale nécessaire.

Ensuite seulement, coder.

Ne jamais casser ce qui fonctionne

Le prototype possède déjà notamment :

- galaxie procédurale ;
- génération par seed ;
- zoom continu ;
- navigation galaxie → système → planète ;
- mouvement orbital ;
- terrain procédural ;
- interactions mobiles ;
- sauvegarde locale.

Ces systèmes constituent une base à préserver.

Une refactorisation ne doit pas devenir une réécriture générale.

Architecture

Toujours séparer :

Données
→ ressources, recettes, bâtiments, technologies, paramètres.

Simulation
→ stocks, production, énergie, temps, transport, règles.

Rendu
→ Canvas, animations, représentation visuelle.

Interface
→ menus, HUD, interactions.

Le rendu ne doit jamais contenir la logique économique faisant autorité.

Une simulation doit autant que possible pouvoir fonctionner et être testée sans Canvas.

Simplicité

Hardgraft est développé par un humain non spécialiste du code avec l'aide d'agents IA.

Le code doit donc rester :

- lisible ;
- explicite ;
- modulaire ;
- documenté lorsque nécessaire ;
- déterministe ;
- facile à modifier.

Éviter :

- abstractions excessives ;
- architectures complexes sans besoin immédiat ;
- frameworks inutiles ;
- dépendances externes ;
- fichiers gigantesques ;

- valeurs d'équilibrage dispersées dans le code.

Lorsqu'une solution simple permet 90 % du résultat recherché, privilégier la solution simple.

Développement data-driven

Les éléments de gameplay susceptibles d'être équilibrés ou étendus doivent être configurables.

Par exemple :

- ressources ;
- recettes ;
- coûts ;
- temps de construction ;
- capacités ;
- production ;
- consommation énergétique ;
- paramètres des technologies.

Ne pas hardcoder ces valeurs directement dans le rendu ou dans des conditions éparpillées.

Scope

Le jeu final prévoit beaucoup de systèmes complexes.

Ils ne doivent PAS être implémentés avant d'être nécessaires.

Notamment, ne pas anticiper :

- IA autonome complexe ;
- politique galactique ;
- rébellions ;
- économie avancée ;
- civilisations complètes ;
- mégastructures ;
- automatisation galactique ;
- système complet de Contexte ;
- data avancée ;
- milliers d'artefacts.

Préparer éventuellement les interfaces nécessaires, mais ne pas coder la mécanique elle-même.

Priorité actuelle du projet

La première grande cible est :

une planète réellement amusante à exploiter.

Le joueur doit pouvoir progressivement :

- construire ;
- extraire ;
- produire ;
- créer une usine ;
- organiser des blocs ;
- observer les stocks et les goulots ;
- alimenter le système en énergie ;
- voir les ressources circuler ;
- développer sa planète ;
- découvrir de nouvelles possibilités.

La complexité galactique vient ensuite.

Travail par étapes

Pour chaque mission :

1. effectuer uniquement le scope demandé ;
2. ajouter ou modifier les tests correspondants ;
3. exécuter tous les tests pertinents ;
4. vérifier mobile et desktop lorsque l'interface est concernée ;
5. ne pas commencer spontanément la mission suivante.

Si une tâche révèle qu'une modification architecturale supplémentaire est nécessaire, expliquer pourquoi avant de l'étendre fortement.

Travail multi-agent

Ne jamais faire modifier simultanément les mêmes fichiers importants par plusieurs agents.

Pour des tâches parallèles :

- attribuer des zones de code distinctes ;
- définir clairement les dépendances ;

- intégrer les résultats avant de commencer la vague suivante.

Le rôle d'un agent n'est pas de « compléter le jeu », mais de produire une brique fiable destinée aux agents suivants.

Tests

Toute mécanique de simulation doit disposer de tests automatisés.

Tester notamment :

- conservation des ressources ;
- limites de stocks ;
- production ;
- blocages ;
- construction ;
- remboursements ;
- déterminisme ;
- transitions d'état.

Ne pas se contenter d'une validation visuelle.

À la fin de chaque tâche

Toujours fournir un rapport court contenant :

Réalisé

Ce qui a été ajouté ou modifié.

Fichiers

Liste des principaux fichiers modifiés/créés.

Tests

Tests exécutés et résultat.

Limites

Ce qui reste volontairement non implémenté.

Risques

Dette technique ou points nécessitant une surveillance.

Étape suivante

Prochaine étape logique, sans l'implémenter sauf demande explicite.

Principe final

Ne cherche pas à impressionner par la quantité de code.

Cherche à produire une base que le prochain agent pourra comprendre en quelques minutes.

Une petite mécanique propre, testée et agréable vaut mieux que cinq systèmes partiellement implémentés.